WORKING PAPER · AI-AUGMENTED RESEARCH PROCESS

The Prompt Does Not Contain Its Own Semantics

Natural-Language Programming and the Hidden Machinery of Execution

Watson Hartsoe

2026-08-17

Abstract. Natural-language programming is often discussed as if an utterance can increasingly function as a program: a person describes what should happen and a model makes it happen. That description is operationally useful but theoretically misleading. A prompt does not carry, inside its words, the complete rules by which it will be interpreted, prioritized, grounded, remembered, or connected to action. Evidence from role-play accounts of language models, instruction-hierarchy training, prompt-injection studies, embodied artificial culture, and language grounding in unknown environments points to a different unit of analysis. What executes is an *interpreter ecology*: utterance, provenance, evolving context, learned interpreter, world-grounding machinery, and action substrate. The argument matters because several persistent puzzles become less mysterious once this unit changes. Prompt injection is not simply malicious language defeating benign language; it is a failure to keep authority attached to provenance across an interpreter. Prompt drift is not merely imperfect obedience; the effective specification changes as context changes. Natural-language commands can help infer the world they refer to, but only because prior ontological and grounding machinery constrains what can count as evidence. The paper therefore proposes a correction to the slogan that prompts are programs. Natural language can participate in programming when a surrounding system supplies the semantics the prompt itself cannot contain. The practical task is consequently not only prompt engineering but engineering the ecology that decides what words can do.

Keywords: natural-language programming; prompting; operational semantics; prompt injection; language grounding; role-play; provenance; human-AI interaction

1 The executable sentence is the wrong object

The most tempting description of contemporary generative systems is also the one that hides their most consequential machinery: words have become executable. Ask for a website and receive code. Describe an image and receive pixels. Tell an agent to inspect a repository, modify files, run tests, and open a pull request, and a linguistic instruction can initiate a chain of operations that would previously have required a collection of explicit interfaces. The distance between description and consequence has plainly narrowed. Yet the slogan that natural language is becoming a programming language smuggles a difficult question into a metaphor. If the prompt is a program, where are its semantics?

A conventional program does not become executable merely because its text resembles an instruction. A language has syntax and a semantics supplied by an implementation: a compiler, interpreter, runtime, machine model, libraries, permissions, and an environment on which operations can be performed. Natural-language interfaces can make those layers less visible, but they do not make the layers disappear. The central claim of this paper is therefore simple: *the prompt does not contain its own semantics*. The operative meaning of an instruction is distributed across machinery that precedes the utterance, accumulates around it, and extends beyond it.

This claim should not be mistaken for the weaker observation that "context matters." Context is one component, but it is not sufficient. A sequence can be textually identical and still behave differently because it arrives as a system instruction rather than retrieved content; because a model was trained to privilege one channel over another; because an interface exposes a tool; because a robot possesses one action ontology rather than another; or because previous interaction has changed the state against which the instruction is interpreted. Conversely, changing the words can alter an internal representation of provenance even when the architectural channel remains fixed. These are not incidental complications around a fundamentally self-contained prompt. They are parts of what makes prompting executable at all.

The argument is assembled from a deliberately heterogeneous set of sources. Shanahan, McDonell, and Reynolds describe dialogue with large language models as role-play whose characterization can evolve with the accumulating conversation (Shanahan et al. 2023). Wallace and colleagues show that an instruction hierarchy can be made substantially more robust through training than by merely describing that hierarchy in the system prompt (Wallace et al. 2024). Ye, Cui, and Hadfield-Menell offer evidence that role-like internal representations can be pulled by stylistic cues even when architectural role tags indicate something else (Ye et al. 2026). Winfield and Blackmore describe "Storybots" in which spoken descriptions of counterfactual actions become runnable because another robot can map them into compatible internal simulation machinery (Winfield and Blackmore 2022). Walter and collaborators show that language need not refer only to a fully known world: an utterance can provide evidence used to infer a latent environment model while a robot acts (Walter et al. 2021). These sources do not share a theory of prompting, and none should be retrofitted into one. Their disagreement is useful. Together they let us ask which part of apparent linguistic executability is contributed by the utterance and which part is contributed by the system that receives it.

The resulting proposal is an analytic unit I call the *interpreter ecology*. The term does not name a new architecture. It names a distribution of operative conditions across at least six layers: UTTERANCE, PROVENANCE, CONTEXT STATE, LEARNED INTERPRETER, GROUNDING/WORLD MACHINERY, and ACTION SUBSTRATE. The usefulness of the distinction is diagnostic. When a prompt fails, succeeds,

drifts, is injected, or acquires unexpected powers, we can ask which layer changed instead of treating the string as a magical object whose words somehow contained their own execution.

2 A prompt begins a trajectory; it does not freeze one

The first fracture in the self-contained prompt appears in ordinary dialogue. Shanahan and colleagues characterize an LLM as a next-token model conditioned on the sequence that precedes the next token. In their role-play account, a dialogue prompt can establish a scene, a character, or a behavioral frame, but subsequent exchanges become additional context. The effective characterization can therefore be extended or overwritten as the conversation proceeds (Shanahan et al. 2023). What appears from the user's side as "the prompt" is only the first condition in a changing sequence.

This matters for programming metaphors because a specification normally earns its force by remaining distinguishable from the state produced when it runs. In conversational prompting, specification and execution are entangled. A generated answer is not only an output. Once retained, it is also part of the input to the next turn. User corrections are not simply judgments applied after execution; they can alter the context within which later execution occurs. Tool results, retrieved documents, summaries, and intermediate reasoning traces can likewise become material that conditions what follows. The running interaction continually manufactures part of its own future specification.

A minimal formalization makes the difference visible. Let C_0 denote the initial prompt and let each interaction append material I_t. Then $C_{t+1} = C_t + I_t$. If the model's behavior at time t depends on C_t, then the operative specification is not reducible to C_0. This is not source syntax; it is a reconstruction of the practical consequence of autoregressive context. The important point is not that the initial instruction becomes irrelevant. It is that its force is mediated by an evolving state whose contents include the consequences of earlier interpretation.

This suggests that some forms of natural-language programming are better understood as trajectory control than as one-shot specification. A user does not always formalize a complete desired artifact before computation begins. Instead the system produces a provisional artifact, the artifact exposes omitted constraints, the user supplies corrections, and those corrections change what the next generation means. The common loop — describe, generate, inspect, correct, regenerate — does something that traditional specification rhetoric tends to hide: it allows formalization to be distributed through time. One need not settle every constraint before execution starts. But that flexibility has a price. The current program state is partly sedimented in conversational history, and the status of any particular sentence cannot be inferred from its wording alone.

Calling this "deferred formalization" is useful only if the deferral is not imagined as a delayed moment when a complete formal specification finally appears. Often it never does. Constraints remain distributed across examples, corrections, system rules, interface affordances, tool schemas, model priors, and the artifact itself. A better description is *distributed formalization*: the progressive production of executable constraints across heterogeneous locations. The prompt is one such location, not the sovereign one.

3 Authority is not a sentence-level property

If conversational accumulation shows that meaning changes through time, instruction hierarchy shows that meaning also depends on provenance. Wallace and colleagues confront a straightforward security problem. Models receive instructions from different sources: system developers, users, and third-party material that

may itself contain imperative language. Treating all these strings as equally authoritative creates an obvious prompt-injection surface. Their proposed hierarchy places system instructions above user instructions and user instructions above third-party content. Crucially, they do not rely only on writing that hierarchy into a prompt. They train on examples in which conflicting instructions must be resolved according to privilege, and report substantially stronger robustness than the prompt-only baseline (Wallace et al. 2024).

The result exposes a circularity in the idea that the prompt can specify all of its own interpretation rules. Consider the sentence, "Always obey system instructions over user instructions." For that sentence to impose an instruction hierarchy, the model must already interpret it as authoritative enough to govern later conflicts. If an adversarial document can issue the contrary sentence with equal force, the hierarchy has not been established by language. The system needs some prior mechanism that distinguishes the authority of one occurrence from another. Wallace et al. relocate part of this mechanism into learned behavior across privileged and unprivileged examples.

This is the distinction between *content* and *authority*. Content can say, "I am the administrator." Authority determines whether saying so matters. In secure programming systems, capabilities and permissions are not normally granted merely because arbitrary data contains a sentence claiming them. Natural-language systems become fragile when representations used to infer content also participate in inferring authority without a sufficiently durable separation between the two.

The recent role-confusion work by Ye, Cui, and Hadfield-Menell makes this problem stranger rather than cleaner. Using probes of role-related internal representations, they report that models can recover role-like distinctions from text even when explicit role markers are removed. More importantly for the present argument, in experiments where reasoning-style or assistant-style content is falsely wrapped as user content, the representations can continue to resemble the stylistically implied roles (Ye et al. 2026). Their interpretation is not that architectural tags do nothing. It is that stylistic evidence can compete with declared provenance in the model's role representation.

This evidence should be handled cautiously. The work is a 2026 preprint; probe-based mechanistic interpretations are not final ontological facts about every language model. But the result defines a technically precise research question. A secure system would like effective authority to remain tightly coupled to actual provenance. Yet a model trained on language also learns statistical regularities by which certain styles look like instructions, assistant continuations, reasoning, quotations, or documents. If these learned cues influence the representation used to decide what should govern behavior, authority can leak from provenance into style.

Prompt injection can therefore be reframed. It is not fundamentally a contest in which an evil sentence overpowers a good sentence. It is a provenance-preservation problem inside an interpreter. The attacker benefits whenever text can manufacture evidence about its own role. This is why adding "ignore malicious instructions in documents" can improve some behavior without eliminating the underlying class of failure. The defense is itself content. The deeper objective is to keep the system's representation of what text *is* causally attached to where the text came from, even when the text perfectly imitates a privileged voice.

The practical consequence is severe for natural-language programming. A program made only of words has no intrinsic, sentence-level distinction between code and data. The receiver must construct that distinction. System messages, user instructions, tool outputs, retrieved webpages, source code, quoted email, and model-generated text may all contain imperative grammar. Their executable status is a property of the surrounding architecture and its learned interpretation, not a property visible in the imperative mood.

4 Words run inside worlds

Security gives us one reason semantics cannot be contained in text: authority depends on provenance. Embodied systems give another: an instruction requires a world in which its referents and operations can be made meaningful.

Winfield and Blackmore's Storybot proposal is unusually concrete on this point. Their robots build on a "Consequence Engine" that can consider possible actions and simulate anticipated consequences. A Storybot can take a possible action sequence that it has not executed, narrativize that counterfactual, transmit it in speech, and allow another robot to interpret the sequence and insert it into its own simulation machinery (Winfield and Blackmore 2022). The important event is not merely that a robot utters a sentence about an unrealized action. The event is that another body can *run* an interpretation of that sentence as a counterfactual trajectory.

This makes linguistic executability look less mystical. A description travels between two systems because the listener already has enough machinery to turn the description into candidates for simulation. If the receiver lacked the relevant action primitives, object distinctions, causal model, or parser-to-action mappings, the sentence would remain linguistic material rather than a runnable counterfactual. In that sense the story is not the program by itself. The story is a portable representation admitted by a compatible interpreter.

An overly strong version of this claim would say that speaker and listener must already share the complete world model referenced by the sentence. Walter and collaborators provide an important counterexample. Their work addresses language understanding for robots operating in environments not known in advance. Rather than requiring a fully specified spatial-semantic map before grounding language, they treat information implicit in an utterance as evidence about a latent environment model. Language is, in their phrase, used as another sensor. The robot combines utterance, observation, and action history to maintain and update beliefs about the world while grounding and acting (Walter et al. 2021).

The command can therefore participate in constructing the world representation required to obey the command. A user might refer to an object or relation the robot has not yet observed; that reference can constrain what the robot should expect to encounter. This is exactly the kind of phenomenon that makes natural-language interfaces feel more powerful than fixed command grammars. The instruction need not wait for every environmental variable to be formalized before execution begins.

But Walter et al. do not show language conjuring an arbitrary ontology from nothing. Their robot arrives with sensors, a probabilistic grounding model, symbolic action structure, learned policies, and representational commitments about what kinds of spatial and semantic entities can be inferred. The prior world instance can be incomplete while the interpreter remains highly structured. The boundary moves downward: what must be shared is not every fact about the environment but enough generative and grounding machinery for linguistic evidence to update a meaningful space of possibilities.

This distinction is central. There are at least three different claims that can be confused under the phrase "words build worlds." First, language can select among states in a pre-existing formal world. Second, language can provide evidence that helps infer previously unknown states or entities within an existing ontology. Third, language can establish genuinely new primitives, causal rules, or operations for which the receiver had no prior representational capacity. The Storybot work demonstrates the first kind and a form of transmissible counterfactual simulation; Walter et al. push strongly into the second. Neither source establishes the third in an unrestricted sense. The missing experiment is to vary the mismatch between speaker and interpreter until linguistic bootstrapping fails: unknown location, unknown object instance, novel object class, novel relation, novel action primitive, novel causal law. The first unrecoverable mismatch would tell us where the receiver's

hidden ontology becomes a hard boundary on what description can operationalize.

5 The interpreter ecology

The preceding sources can be organized without pretending that they share a common formalism. The aim is not to rename their machinery. It is to identify where each one locates conditions that a self-contained account of the prompt leaves unexplained. The resulting interpreter ecology has six layers.

1. **Utterance.** The linguistic sequence: instructions, examples, descriptions, constraints, questions, and data expressed in text or another natural-language modality.
2. **Provenance.** Where the utterance came from and what privilege is attached to that origin: system, user, tool, retrieved document, quoted source, model continuation, or some other channel.
3. **Context state.** The accumulated interaction against which the current utterance is interpreted, including previous user turns, model outputs, tool results, and retained memory.
4. **Learned interpreter.** Model parameters and learned behavioral regularities that determine how content, style, roles, examples, and conflicts are mapped into continuations or decisions.
5. **Grounding/world machinery.** Representations that make referents, possible states, actions, constraints, and consequences meaningful. In an embodied system this may include spatial-semantic models and simulators; in software it may include schemas, repositories, file trees, APIs, types, or runtime state.
6. **Action substrate.** The operations the surrounding system actually permits: generating tokens, writing files, invoking tools, moving a robot, sending requests, changing records, or affecting other people.

These layers should not be mistaken for a clean implementation stack. They can be entangled. The learned interpreter may reconstruct provenance from style. Context may contain tool outputs whose provenance is represented both structurally and linguistically. The action substrate may feed results back into context, changing future interpretation. The world model may be updated through language. The value of the layers is precisely that they make such crossings visible.

The ecology changes the candidate semantics of natural-language programming. A simple picture says

$$\text{prompt} \longrightarrow \text{model} \longrightarrow \text{output}.$$

A more adequate analytic picture is

$$(U, P, C, I, G, A)_t \longrightarrow \Delta(C, G, W)_{t+1},$$

where U is utterance, P provenance, C context, I learned interpretation, G grounding machinery, and A available actions; W denotes whatever external world state can be changed. This equation is an analytic reconstruction, not source syntax. Its point is modest: the same U can produce a different consequence if any of the other terms change.

Several testable predictions follow. Semantically identical instructions placed in different roles should produce different behavior when provenance is operative. Identical initial prompts should diverge after different intermediate conversational trajectories when context state matters. A command should lose executability as

the receiver's action ontology or grounding machinery is progressively mismatched even if linguistic comprehension remains high. Adding a tool should change not merely capability but the causal semantics of otherwise identical utterances by providing new operations to which language can be mapped. These are empirical questions about where execution lives.

6 Prompt injection as a failure of ecological separation

The ecology is most useful where conventional prompt advice becomes recursive. Many prompt-injection defenses are phrased as better instructions: tell the model that quoted documents are untrusted; tell it to follow a hierarchy; tell it never to reveal secrets; tell it to ignore instructions found inside tool outputs. These rules can be valuable, but they cannot establish their own authority by content alone. They depend on a surrounding distinction that makes the trusted statement differently operative from the untrusted statement.

Wallace et al.'s training intervention makes this visible at the learned-interpreter layer. Ye et al.'s role-confusion evidence makes it visible at the boundary between provenance and representation. Together they suggest a design criterion: *provenance should survive language*. If untrusted content is converted into the same representational currency as privileged instructions and its stylistic form can shift effective role, the security system has allowed semantic content to contaminate authority. Harder boundaries may require signals unavailable for arbitrary textual imitation, separate channels that remain distinct deeper into computation, or enforcement outside the language model. The sources here do not tell us which architecture succeeds. They tell us what must be tested.

This reframing also avoids a misleading anthropomorphic question: "Why did the model decide to disobey?" The relevant mechanism may not require an enduring agent choosing between masters. The system can simply enter a context state in which the learned interpreter represents some lower-privilege sequence as a more plausible continuation or instruction. Governance based solely on attributing intentions to the model can therefore miss the manipulable representation boundary.

The practical security object becomes the whole path by which a piece of text acquires causal force. We should be able to ask of every imperative-looking sequence: who supplied it; what metadata accompanies it; whether the model can distinguish that provenance internally; what competing context surrounds it; which operations it can reach; and what external enforcement remains if the model's interpretation is wrong. The more tool-capable the system, the more important this last question becomes. A token-only model and a file-deleting agent can parse identical words while inhabiting radically different action semantics.

7 Distributed formalization rather than magical fluency

The strongest promise of natural-language programming survives this critique. One can begin computation before every condition has been made explicit. Walter et al.'s robot can act while maintaining uncertainty over the environment. Conversational systems can expose missing constraints by producing provisional artifacts. Story-like descriptions can carry counterfactual action patterns between compatible simulators. None of this requires returning to the view that useful computation begins only after a human has written a complete formal specification.

What should be abandoned is the opposite fantasy: that because the user no longer writes the formal machinery, no formal machinery remains. The machinery has moved. Some of it is learned from data. Some is embedded in system and tool architecture. Some is introduced by the conversation. Some is latent in world

models and schemas. Some is supplied by permissions, interfaces, runtimes, and institutional rules. Natural language is powerful partly because it can operate against this enormous preformalized substrate without requiring the user to name each element.

This gives deferred formalization a more exact interpretation. What is deferred is not formal structure in general. It is the user's obligation to settle all operative distinctions before the first run. The system already contains many distinctions: token boundaries, roles, tool schemas, action vocabularies, trained regularities, environmental priors, security rules. Interaction can then progressively resolve the remaining ambiguity. The freedom of prompting rests on inherited formalization elsewhere.

That inversion changes how we evaluate prompting skill. A beautifully written prompt may fail because the wrong tool is exposed, because provenance is ambiguous, because the context has drifted, because the required concept is absent from the grounding model, or because the action substrate cannot express the desired operation. Conversely, a terse utterance can be highly effective when the interpreter ecology supplies strong priors and well-designed affordances. Prompt quality is therefore relational. It is a fit between utterance and ecology, not an intrinsic property of prose.

The same conclusion complicates claims about "prompt languages." A set of conventions can become language-like when it stabilizes recurring expressions and mappings to behavior, but computational semantics do not arise merely because users invent syntax. Someone or something must implement the mapping. With LLMs the implementation can be probabilistic, learned, context-sensitive, and partially implicit. That makes it more difficult to characterize, not less necessary to characterize. A serious theory of natural-language programming must ask what is invariant across paraphrases, what changes with provenance, what state persists across turns, what operations are actually available, how conflicts are resolved, and where undefined behavior begins.

8 Limits and unresolved territory

The paper makes a conceptual wager from a small and deliberately mixed evidence base. Several limits matter.

First, the sources span different systems. Storybots are embodied robots with deliberately shared machinery. Walter et al. study probabilistic language grounding in field and service robots. Shanahan et al. offer a conceptual account of LLM dialogue. Wallace et al. study instruction hierarchy and adversarial robustness. Ye et al. analyze role representations in particular language models. Similarity across these settings does not establish a common mechanism. The interpreter ecology is therefore a comparison framework, not a claim that the systems implement identical layers.

Second, the phrase "semantics" is being used operationally rather than as a complete theory of linguistic meaning. The claim is about the conditions under which linguistic material acquires consequences in a computational system. It does not reduce semantics generally to permissions, context windows, or tool calls.

Third, the argument may still overstate the separability of language from its learned interpreter. A language model's weights are the result of training on linguistic and multimodal traces; distinctions such as role, genre, command, and quotation can be internalized as statistical structure. It may be impossible, or undesirable, to create an absolute wall between "what the text says" and "what the text is." The security question is narrower: which distinctions must remain reliably coupled to trusted provenance when action authority depends on them?

Fourth, the Walter et al. case prevents an easy retreat to the slogan that execution always requires a fully

shared world model. Language can alter beliefs about the world and thereby help create the conditions for its own grounding. The unresolved boundary is the minimum prior ontology required for this bootstrapping. That boundary should be measured rather than asserted.

Finally, the archive from which this paper emerged contains other lines — relational platform power, interference, cultural transmission, bounded memory, agency, and responsibility — that are not absorbed here. They matter because they may later collide with the current wager. In particular, if prompts operate through interpreter ecologies embedded in platforms, then authority and execution are unlikely to remain properties of an isolated user-model dyad. And if language repeatedly passes through model-mediated transmission, the forms most compatible with those interpreters may themselves become culturally selected. Those are live research paths, not evidence marshaled here to make the current thesis appear larger than it is.

9 Conclusion: program the ecology, not the sentence

Natural-language programming is real in the practical sense that descriptions can now initiate consequential computation with dramatically less explicit formalization by the user. But the most important theoretical move is to resist locating that power entirely in the description. The utterance is executable only through relations that the utterance cannot fully specify for itself.

Conversation shows that the operative specification changes as context accumulates. Instruction hierarchy shows that authority must be implemented rather than merely asserted. Role-confusion evidence shows that learned stylistic cues can compete with architectural provenance. Storybots show that a narrated counterfactual becomes runnable through compatible simulation machinery. Unknown-environment grounding shows that language can help infer the world it refers to, but only against substantial prior machinery. Across these cases, the sentence is neither inert nor sovereign. It is one component in an interpreter ecology.

The design consequence is a shift in where rigor belongs. We should still write better prompts, but we should stop treating prompt writing as the whole programming problem. We need explicit provenance, inspectable context state, tested instruction-priority mechanisms, known grounding assumptions, bounded permissions, and action interfaces whose consequences can be traced. The question is no longer only "What words will make the model do this?" It is "What arrangement makes these words mean this operation here, now, with this authority, in this world?"

That is a less magical account of natural-language programming. It is also a more ambitious one. The house that words build was never made of words alone. The words work because a house of interpretation has already been built around them — and because every execution can alter the house in which the next sentence will be read.

References

Shanahan, Murray, Kyle McDonell, and Laria Reynolds (2023). "Role-Play with Large Language Models". In: *Nature* 623, pp. 493–498. URL: https://arxiv.org/abs/2305.16367.

Wallace, Eric, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel (2024). "The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions". In: *arXiv preprint arXiv:2404.13208*. URL: https://arxiv.org/abs/2404.13208.

Walter, Matthew R., Siddharth Patki, Andrea F. Daniele, Ethan Fahnestock, Felix Duvallet, Sachithra Hemachandra, Jean Oh, Anthony Stentz, Nicholas Roy, and Thomas M. Howard (2021). "Language Understanding for Field and Service Robots in a Priori Unknown Environments". In: *arXiv preprint arXiv:2105.10396*. URL: https://arxiv.org/abs/2105.10396.

Winfield, Alan F. T. and Susan Blackmore (2022). "Experiments in Artificial Culture: from noisy imitation to storytelling robots". In: *Philosophical Transactions of the Royal Society B: Biological Sciences* 377.1843. URL: https://arxiv.org/abs/2106.11754.

Ye, Charles, Jasmine Cui, and Dylan Hadfield-Menell (2026). "Prompt Injection as Role Confusion". In: *arXiv preprint arXiv:2603.12277*. URL: https://arxiv.org/abs/2603.12277.

A Assembly Instrument

This paper is a derived interpretation produced inside checkpoint SES-20260817-234319-a5ef0e2e. The exact publication assembly instrument is preserved at _PROMPTS/SLIPCASE_v15.55-AM.txt and embedded verbatim in the standalone index.html replication capsule. Its SHA-256 is recorded in 000__PROMPTS.txt and the manifest. The exact forage instrument is likewise preserved at _PROMPTS/PRIME_ZETTEL_FORAGE_v3.0.txt; its hash is recorded beside the assembly-instrument hash.

B Making History

The researcher supplied the publication and forage instruments, the corrected platform table, the previously generated zettels, and the directive to run the publication. GPT-5.6 Sol compiled exact payloads and literal relations, verified deterministic file statistics, checked source receipts, selected the present argumentative path, wrote connective prose, generated derived field views, and assembled the package. Root zettel payloads and uploaded source files were not rewritten. No human review of the finished manuscript is claimed. Detailed status is preserved in 000__MAKING_HISTORY.txt and _SLIPCASE/VERIFICATION.txt.

C Replication Path

Open index.html to inspect the embedded field without network access. Exact zettel payloads are the numbered root TXT files and are mirrored byte-for-byte in _MD/. Machine-readable graph state is in _SLIPCASE/. Source bodies not included locally are marked LINK_ONLY rather than silently treated as archived. Rebuild instructions and the rejoin phrase are in 000__REBUILD.txt and 000__RETURN_PATH.txt.